

Skills
Network

파이썬에서 루프 소개

예상 소요 시간: 10분

목표

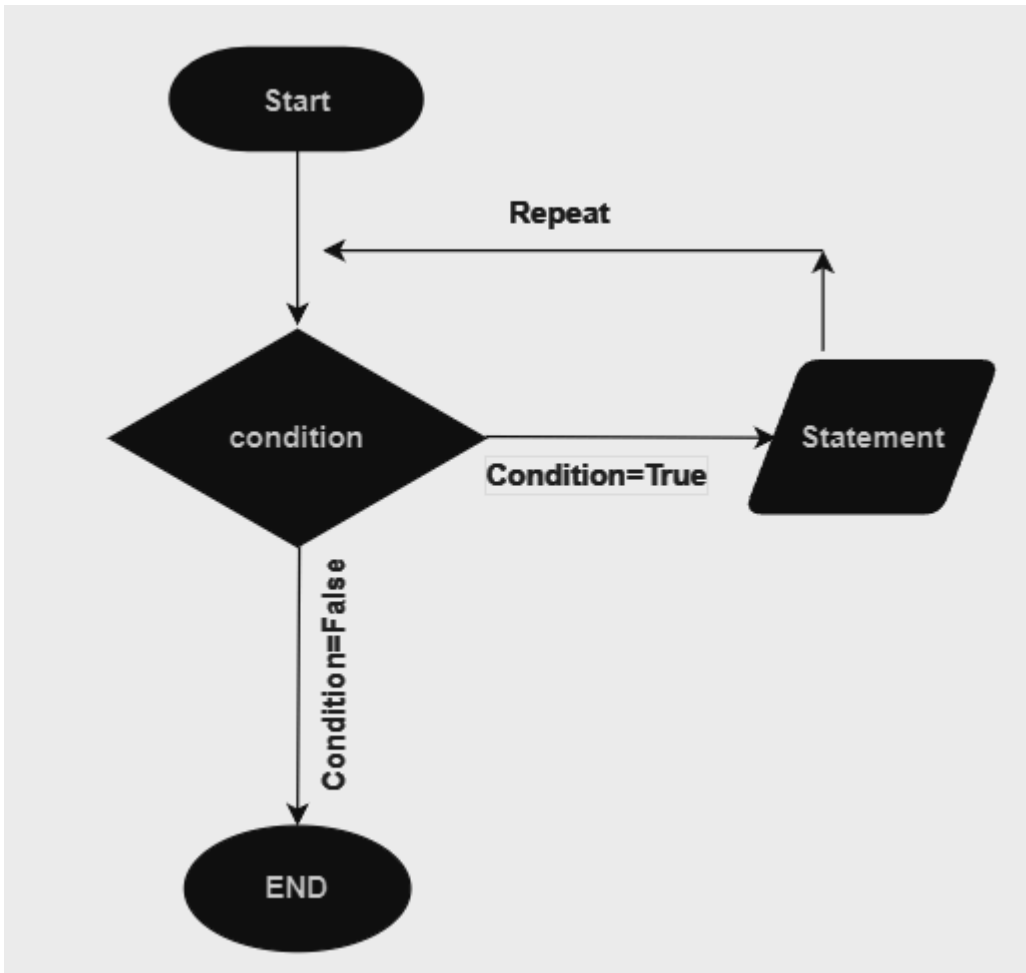
1. 파이썬 루프 이해하기.
2. 루프 작동 방식.
3. 루프의 필요성 배우기.
4. 파이썬의 Range 함수 활용하기.
5. 파이썬의 enumerate 함수 익히기.
6. 조건 작업을 위한 while 루프 적용하기.
7. 적절한 루프 선택 구분하기.

루프란 무엇인가?

프로그래밍에서 루프는 컴퓨터가 무언가를 반복해서 수행할 수 있게 해주는 마법 같은 기술입니다. 당신이 마법사의 조수라고 상상해 보세요. 마법사 친구가 모자에서 토끼를 꺼내라고 요청하지만, 단 한 번이 아니라 멈추라고 할 때까지 계속하길 원합니다. 이것이 루프가 컴퓨터를 위해 하는 일입니다. 루프는 필요한 만큼 명령어 세트를 반복합니다.

루프가 작동하는 방식은?

다음은 Python에서 작동하는 방식입니다:



- **시작:** for 루프는 키워드 for로 시작하며, 그 뒤에 시퀀스의 각 값을 취할 변수를 지정합니다.
- **조건:** 변수 다음에 in 키워드를 지정하고, 루프가 반복할 시퀀스(예: 리스트 또는 범위)를 설정합니다.
- **조건이 참일 경우:**
 1. 루프는 시퀀스에서 첫 번째 값을 가져와 변수를 할당합니다.
 2. 루프 헤더 다음의 들여쓰기된 코드 블록이 이 값을 사용하여 실행됩니다.
 3. 루프는 시퀀스의 다음 값으로 이동하여 모든 값이 사용될 때까지 이 과정을 반복합니다.
- **문장:** 루프의 들여쓰기된 블록 안에 시퀀스의 각 값에 대해 반복할 문장을 작성합니다.
- **반복:** 루프는 시퀀스의 각 값에 대해 코드 블록을 반복하며 더 이상 값이 남지 않을 때까지 계속됩니다.
- **조건이 거짓일 경우:**
 1. 시퀀스의 모든 값이 처리되면 루프는 자동으로 종료됩니다.
 2. 루프가 실행을 완료하고, 프로그램은 루프 다음의 다음 문장으로 계속 진행됩니다.

루프의 필요성

1부터 10까지 세어야 할 때를 생각해 보세요. 수동으로 하는 것은 쉽지만, 백만까지 세어야 한다면 어떨까요? 모든 숫자를 하나씩 입력하는 것은 악몽이 될 것입니다! 이럴 때 루프가 유용합니다. 루프는 컴퓨터가 지

치지 않고 빠르고 정확하게 작업을 반복하도록 도와줍니다.

주요 루프 유형

For 루프

For 루프는 슈퍼히어로의 체크리스트와 같습니다. 프로그래밍에서 for 루프는 리스트의 요소나 범위의 숫자와 같은 시퀀스의 각 항목에 대해 일련의 문장을 반복적으로 실행할 수 있게 해주는 제어 구조로, 효율적인 반복과 작업 자동화를 가능하게 합니다.

for 루프의 구문

```
for val in sequence:  
    # statement(s) to be executed in sequence as a part of the loop.
```

여기 for 루프의 예가 있습니다.

당신이 화가라고 상상해 보세요. 아름다운 무지개를 일곱 가지 색으로 칠하고 싶습니다. 각 색을 하나씩 선택하여 무지개를 칠하는 대신, 마법의 화가 보조에게 그렇게 해달라고 요청할 수 있습니다. 이것이 프로그래밍에서 기본 for 루프가 하는 일입니다.

우리는 색상 목록이 있습니다.

```
colors = ["red", "orange", "yellow", "green", "blue", "indigo", "violet"]
```

색상 이름을 새 줄에 출력하기 위해 for 루프를 사용합니다.

```
for color in colors:  
    print(color)
```

이 예제에서 for 루프는 colors 목록에서 각 색상을 선택하고 화면에 출력합니다. 각 색상에 대해 동일한 코드를 작성할 필요가 없습니다 - 루프가 자동으로 처리합니다!

가끔은 무지개를 그리는 대신, *목표에 도달하기 위한 단계 수를 세고 싶을 수 있습니다*. 범위 기반 for 루프는 목표에 도달하는 데 도움을 주는 친근한 단계 카운터와 같습니다. 다음은 1에서 10까지 세기 위해 for 루프를 사용하는 방법입니다:

```
for number in range(1, 11):  
    print(number)
```

여기서, **range(1, 11)**는 1부터 10까지의 시퀀스를 생성하고, for 루프는 그 시퀀스의 각 숫자를 반복하며 출력합니다. 마치 10걸음을 걷는 것과 같고, 루프가 당신을 안내합니다!

범위 함수

Python의 범위 함수는 루프에서 사용할 수 있는 정렬된 시퀀스를 생성합니다. 하나 또는 두 개의 인수를 받습니다:

- 하나의 인수(예: range(11))가 주어지면, 0부터 주어진 숫자까지(주어진 숫자는 포함하지 않음) 시퀀스를 생성합니다.

```
for number in range(11):  
    print(number)
```

- 두 개의 인수(예: range(1, 11))가 주어지면, 첫 번째 인수부터 두 번째 인수를 포함하지 않는 시퀀스를 생성합니다.

```
for number in range(1, 11):  
    print(number)
```

열거된 For 루프

리스트에서 항목과 그 위치를 모두 추적해야 했던 적이 있습니까? 열거된 for 루프가 당신을 도와줍니다. 항목을 전달할 뿐만 아니라 어디에서 찾을 수 있는지 알려주는 개인 비서를 갖는 것과 같습니다.

다음 예를 고려해 보세요:

```
fruits = ["apple", "banana", "orange"]
for index, fruit in enumerate(fruits):
    print(f"At position {index}, I found a {fruit}")
```

이 루프를 사용하면 과일뿐만 아니라 목록에서의 위치도 얻을 수 있습니다. 마치 마법의 안내자가 각 과일의 위치를 가리키는 것과 같습니다!

While Loops

While 루프는 친구의 잠자리에서 잠 못 이루는 밤과 같습니다. 당신과 친구들이 누군가 잠자리에 들 시간이 되었다고 결정할 때까지 계속해서 유령 이야기를 하고 있다고 상상해 보세요. 누군가 “자자”라고 말하지 않는 한 이야기를 계속합니다.

While 루프는 이와 유사하게 작동합니다 - 특정 조건이 참인 동안 작업을 반복합니다. “야 컴퓨터, 내가 멈추라고 할 때까지 계속 해!”라고 말하는 것과 같습니다.

While 루프의 기본 구문.

```
while condition:
    # Code to be executed while the condition is true
    # Indentation is crucial to indicate the scope of the loop
```

예를 들어, 다음은 1부터 10까지 세기 위해 while 루프를 사용하는 방법입니다:

```
count = 1
while count <= 10:
    print(count)
    count += 1
```

위 코드를 분해해 보겠습니다.

1. **count**라는 이름의 변수가 1로 초기화됩니다.
2. **while** 루프는 주어진 조건이 True인 동안 코드 블록을 반복 실행하는 데 사용됩니다. 이 경우 조건은 **count <= 10**으로, count가 10 이하인 동안 루프가 계속됩니다.
3. 루프 내부:
 - **print(count)** 문은 count 변수의 현재 값을 출력합니다.
 - **count += 1** 문은 count의 값을 1만큼 증가시킵니다. 이 단계는 count가 10을 초과할 때 루프가 결국 종료되도록 보장합니다.
4. 루프는 count <= 10 조건이 만족되는 한 계속 실행됩니다.
5. 루프는 print 문이 루프 블록 내에 있고 각 반복 동안 실행되므로 1부터 10까지의 숫자를 순차적으로 출력합니다.
6. count가 11에 도달하면 count <= 10 조건은 False로 평가되어 루프가 종료됩니다.
7. 코드의 출력은 1부터 10까지의 숫자가 각기 다른 줄에 출력됩니다.

루프 흐름

for 루프와 while 루프는 각각의 특별한 동작이 있지만, 패턴을 따릅니다:

- **초기화:** 시작점이나 조건과 같은 것을 설정합니다.
- **조건:** 루프가 계속 진행되어야 할 때와 멈춰야 할 때를 결정합니다.
- **실행:** 루프 안에서 작업을 수행합니다.
- **업데이트:** 앞으로 나아가기 위해 시작점이나 조건을 변경합니다.
- **반복:** 조건이 더 이상 참이 아닐 때까지 루프는 2단계로 돌아갑니다.

각 용도의 사용 시기

For Loops: for 루프는 반복 횟수를 미리 알고 있고 시퀀스의 각 요소를 처리하고자 할 때 사용합니다. 이들은 길이가 알려진 컬렉션과 시퀀스를 반복하는 데 가장 적합합니다.

While Loops: while 루프는 특정 조건이 참인 동안 작업을 반복적으로 수행해야 할 때 사용합니다. while 루프는 반복 횟수가 불확실하거나 특정 조건이 충족될 때까지 기다려야 하는 상황에서 특히 유용합니다.

요약

이번 코딩 모험에서는 파이썬의 루프를 탐험했습니다. 루프는 우리가 지치지 않고 반복적으로 작업을 수행할 수 있게 도와주는 특별한 도구입니다. 우리는 두 가지 유형의 루프를 만났습니다: **"for 루프"**와 **"while 루프."**

For 루프는 작업을 순서대로 반복할 수 있게 해주는 도우미와 같았습니다. 우리는 색칠을 하고, 숫자를 세고, 목록에서 물건의 위치를 알려주는 도우미를 얻기도 했습니다. For 루프는 우리의 작업을 더 쉽게 해주고 코드를 더 깔끔하게 만들어 주었습니다.

While 루프는 규칙이 참인 동안 계속해서 무언가를 하는 탐정과 같았습니다. 그들은 우리가 단계를 밟고, 숫자를 추측하고, 피곤할 때까지 작업을 할 수 있도록 도와주었습니다. While 루프는 우리가 멈추라고 할 때까지 멈추지 않는 스마트한 도우미와 같았습니다.

Author(s)

[Akansha Yadav](#)

변경 이력

날짜	버전	변경자	변경 설명
2023-21-08	1.0	Akansha Yadav	읽기 파일 생성